AESC / NeuroField v2: Two-Speed Memory for Recurrent Language Models

Giridhari Karmakar
Independent researcher · Project AERIS_64
giriisdev@gmail.com
https://github.com/giridhari-code/AERIS_64

October 2026 · Technical research paper (preprint-style)

Abstract

We present **NeuroField (AESC v2)**, a recurrent sequence model that replaces full self-attention with (1) a **predictive dendrite** over a short causal window, (2) a **k-step neural field** that refines hidden state with local coupling and memory readout, (3) **top-k skill routing**, and (4) an explicit **two-speed memory**: fast weights updated by a surprise-gated, decay-consistent delta rule, and slow weights updated by surprise-weighted replay into a content-addressed ring. The shipped implementation uses a **parallel / looped** forward path. We give complete forward equations, diagrams, safety behaviour, training objectives, and an honest status report.

Keywords: recurrent language models, fast weights, delta rule, neural field, skill routing, two-speed memory, AESC, AERIS

1. Introduction

Transformers dominate language modelling through stacked causal self-attention. Their strength is global context mixing; their cost is quadratic attention. AESC / NeuroField explores recurrence, fast weights, and modular skills as an alternative inductive bias. The product shell is **AERIS_64**. Contributions: (i) full forward specification including decay-consistent delta rule; (ii) parallel/looped path with causality constraints; (iii) gate-based safety freeze; (iv) diagrams and limitation list for follow-on work.

2. Related work (brief)

Fast weights and delta-rule memories; neural fields / iterative state maps; mixture-of-experts routing; complementary learning systems (slow/fast memory). AESC is not claimed to supersede Transformers.

3. Architecture

3.1 High-level dataflow

Figure 1. NeuroField / AESC v2 high-level dataflow

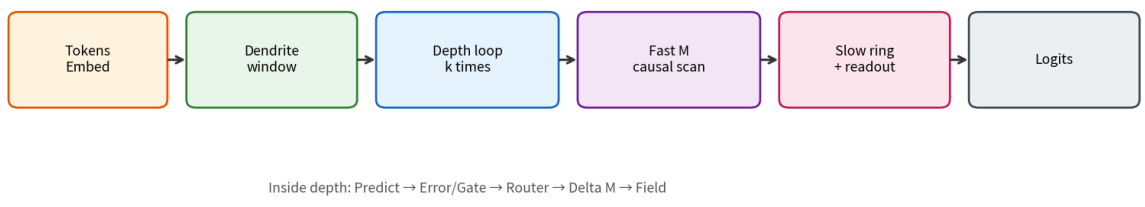


Figure 1. End-to-end dataflow from tokens to logits.

```
token ids -> Embed -> DendriteWindow -> for depth j=1..k:
    Predict+error, Gate, Router, Fast delta scan, FieldStep
  -> Slow ring + Readout -> logits
```

3.2 Embedding

```
X_t = Embed(token_t)  in R^d
```

3.3 Predictive dendrite

```
d_win_t = tanh( sum_{j=0}^{W-1} c_j * x_{t-j} + b )
d_t     = d_win_t + gamma * e_t
```

3.4 Prediction, error, surprise

```
x_hat_{t+1} = f_p(h_t)
e_{t+1}     = stopgrad(x_{t+1}) - x_hat_{t+1}
S_{t+1}     = stopgrad( mean |e_{t+1}| )
S_bar_t     = rho * S_bar_{t-1} + (1-rho) * S_t
S_tilde_t   = S_t / (S_bar_t + eps)
```

3.5 Neuromodulatory write gate

```
g_t = sigmoid( a_s*(S_tilde_t - 1) + a_c*mean(|h_t|) + b + delta_t )
g_t = min(g_t, g_max)
If memory_frozen: g_t = 0  => fast M only decays.
```

3.6 Neural field

```
h <- (1-zeta)*h + zeta*LayerNorm(
        alpha*h + beta*( d_t + R(tanh(h)) + W_m*r_{t-1} ))
```

3.7 Skill router

```
pi = softmax(W_r*h);  w = softmax(top-k(pi))
w_STE = w + pi - stopgrad(pi)
o_t = sum_i w_STE_i * Op_i(h)
```

3.8 Fast memory (Change 5 delta rule)

Figure 2. Two-speed memory

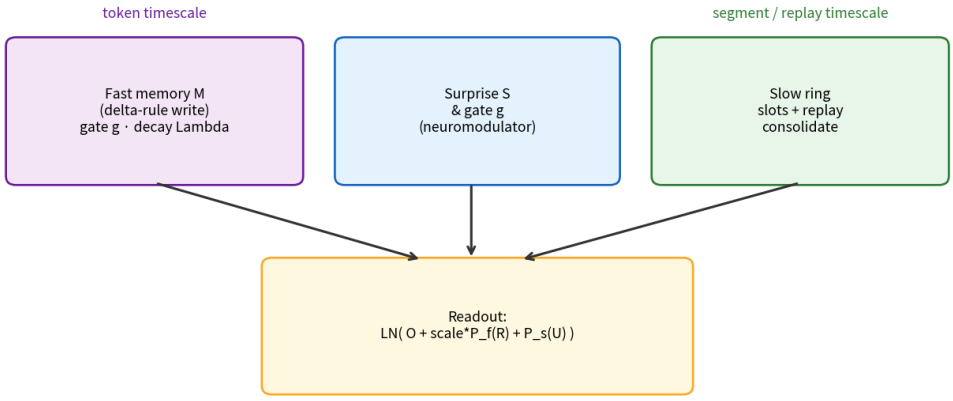


Figure 2. Fast matrix memory vs slow ring consolidation.

```
k_t = normalize(W_k*d_{t-1});  v_t = W_v*o_t
Lambda = diag(sigmoid(theta));  M_dec = Lambda*M_{t-1}
M_t = M_dec + g_t * outer(k_t, v_t - M_dec^T*k_t)
r_t = M_t^T * normalize(W_q*d_t)
If g_t=0: M_t = Lambda*M_{t-1}
```

3.9 Slow memory and readout

```
u_t = SlowRead(d_t; ring)
z_t = W_o * LayerNorm( o_t + s*P_f(r_t) + P_s(u_t) )
logits_t = z_t
```

3.10 Parallel / looped production path

Figure 4. One depth pass (loop body)

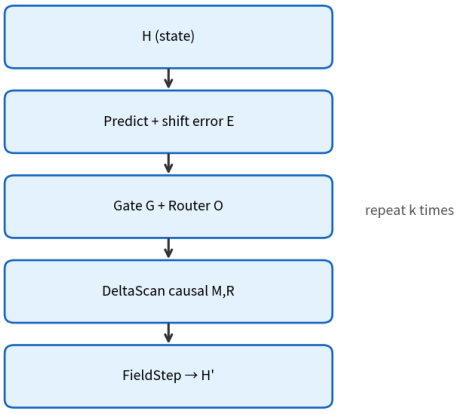


Figure 4. Body of one depth pass (repeated k times).

```
X=Embed(tokens); D_win=DendriteWindow(X); H=D_win
for j in 1..k:
  E = stopgrad(X)-shift(Predict(H)); D = D_win+gamma*E
  O=Router(H); G=Gate(E,H); M,R=DeltaScan_causal(D,O,G)
  H=FieldStep(H,D,shift(R))
logits=Readout(O,R,U_slow)
```

4. Causality

Figure 3. Structural causality (no attention mask)

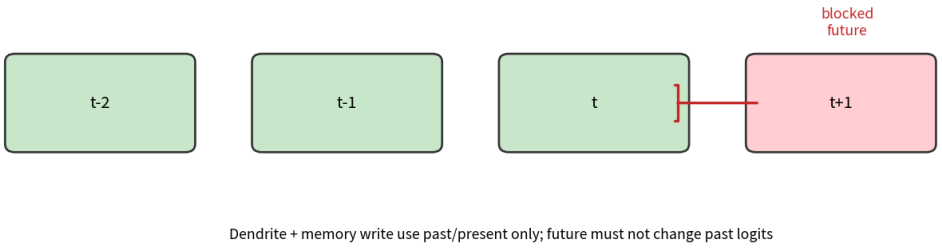


Figure 3. Future tokens must not affect past logits.

No triangular attention mask. Causality via dendrite window, shifted error, ordered memory writes, and shifted reads. Leakage tests: change x_{t+1} must not change logits at indices <= t.

5. Training objective

```
L = CE(logits, targets; label_smoothing)
  + lambda_err * mean(||E||^2)
  + lambda_bal * routing_balance
```

AdamW, grad clip, warmup, optional cosine/Noam LR, dropout, DDP via torchrun.

6. Safety observability

memory_frozen forces g_t=0 so fast memory only decays—runtime control without destroying weights.

7. Implementation status

Open-source AERIS_64 (PyTorch): train, safetensors, FastAPI serve. Small runs show loss drop and causal checks when data is non-empty. Not a competitive large-LM claim. Throughput below similar-width Transformers due to causal scan + depth k.

8. Limitations and future work

1. Data scale / SFT quality.
2. Training throughput.
3. Gate collapse.
4. Train–decode parity under refactors.
5. Multi-seed eval vs Transformer baselines.
6. Theory of depth-k delta memory.
7. Broader safety policy.

9. Conclusion

NeuroField / AESC v2 is a complete two-speed-memory recurrent design with equations, diagrams, working code, and clear limits. Quality at scale needs data and compute; the core specification stands.

Author information

Giridhari Karmakar
Email: giriisdev@gmail.com
Project: AERIS_64 / NeuroField (AESC)
Code: https://github.com/giridhari-code/AERIS_64

Preprint-style technical paper for archival citation. Not a claim of peer-reviewed acceptance.

Figure list

Fig. 1 Pipeline · Fig. 2 Two-speed memory · Fig. 3 Causality · Fig. 4 Depth-pass body